Hinglish Audio Turn Detection

Predicting whether a speaker has finished their turn, directly from audio

Contents

1 What I was trying to solve **2**
2 Turn detection is not VAD **2**
3 Where this fits in a voice AI system **2**
4 Audio-based vs. text-based turn detection **2**
5 What I built **3**
6 Dataset **4**
7 Data engineering **4**
8 Audio preprocessing **4**
9 Why Whisper Tiny **4**
10 The model **4**
11 Filler words **5**
12 Training strategy **6**
13 Train / validation / test **6**
14 Why F1 **6**
15 Results **6**
16 Demo **7**
17 A better product-level formulation **8**
18 What I would build next **8**
19 Final architecture **9**
20 Conclusion **9**

1 What I was trying to solve

In a voice AI system, knowing **when to respond** is as important as knowing **what the user said**.

Consider:

> "I want to book a flight to… um…"

The user has paused, but they are clearly not finished.

Now consider:

> "I want to book a flight to Delhi."

The user may stop speaking for exactly the same reason — silence — but this time the system should respond.

This is the problem I wanted to solve: **given the user's recent audio, can we predict whether the user has finished speaking or is likely to continue?** This is called **turn detection**.

2 Turn detection is not VAD

A useful distinction is:

VAD asks: *"Is the person speaking right now?"* It mainly detects speech vs. silence.

Turn detection asks: *"Has the person finished their thought?"*

A user can stop speaking for a short time without actually finishing their sentence — for example, "I need to book a flight to…" followed by a pause. VAD may see silence, but a good turn detector should understand that the user probably wants to continue. This is why turn detection needs more information than just the amount of silence.

3 Where this fits in a voice AI system

VAD and turn detection work together. VAD detects that the user has paused; turn detection then looks at the audio and decides whether that pause means "I am finished" or "I am still thinking." This is also how existing Smart Turn systems are designed — VAD can trigger analysis, and the turn model uses the larger audio context to make the final decision.

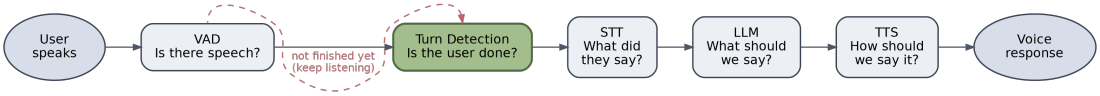


Figure 1: VAD and turn detection work together: VAD flags a pause, and the turn model decides what it means before the pipeline moves on to STT, the LLM, and TTS.

4 Audio-based vs. text-based turn detection

There are two ways to approach this problem.

Audio-based: the turn detector looks directly at the user's voice. The benefit is that we can potentially make the turn decision before waiting for a complete STT result.

Text-based: the turn detector works on the transcript. This gives the model access to the actual words, but it also means we have already paid the STT latency.

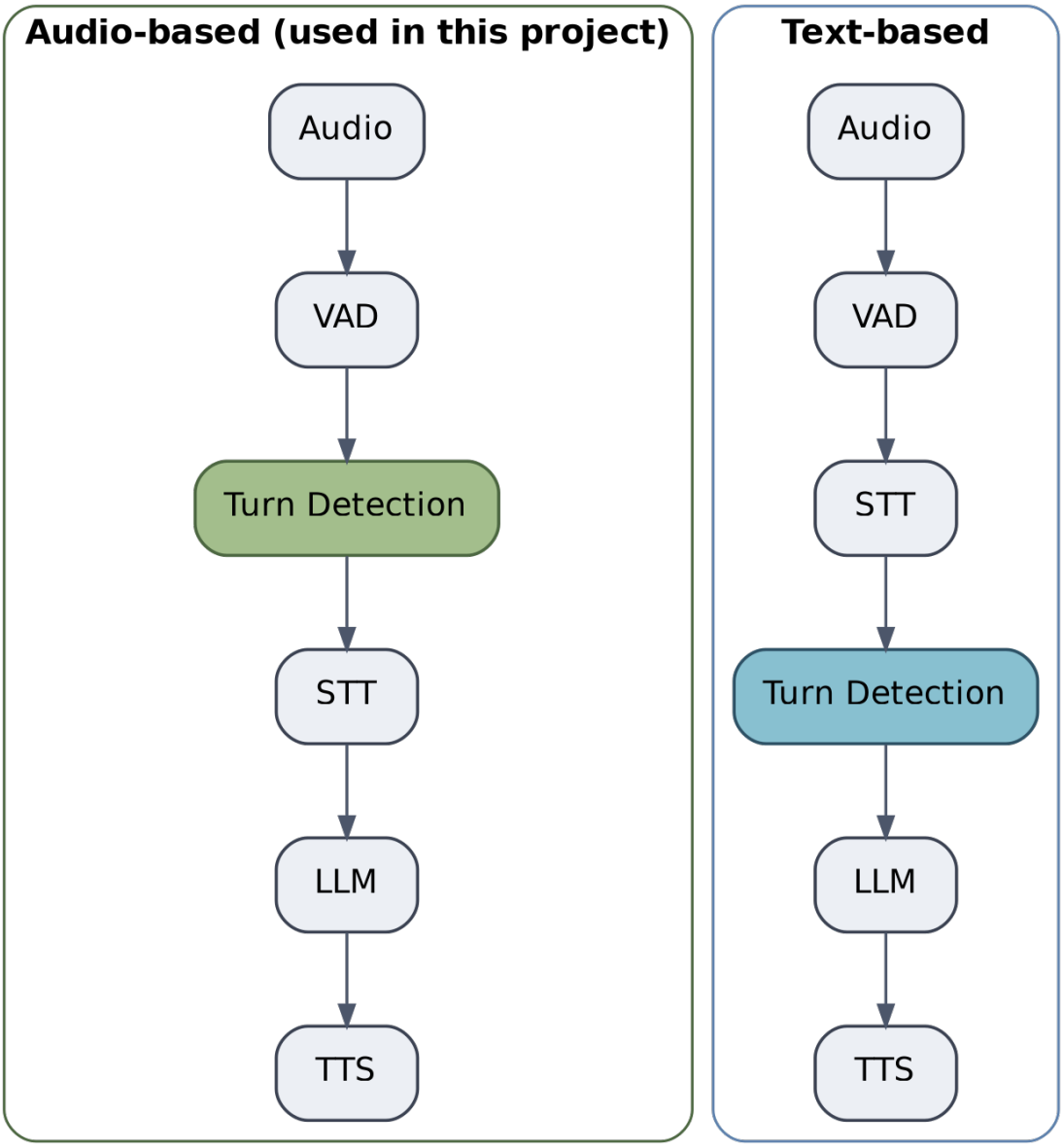


Figure 2: Audio-based turn detection runs before STT and avoids paying its latency; text-based detection runs after STT and gets the transcript instead.

For this challenge, I focused on **audio-based turn detection**.

5 What I built

My first version treats the problem as a binary classification problem: given the user's audio, output `P(end of turn)`. For example, `P(end of turn) = 0.91` means the model is confident that the user has finished.

6 Dataset

I used the public `pipecat-ai/smart-turn-data-v3.2-train` dataset. It contains audio together with turn-related labels: whether the turn is complete, whether there is a filler in the middle, and whether there is a filler near the end.

For this experiment, I selected:

Language	Samples
Hindi	20,000
English	20,000
Total	**40,000**

I intentionally kept Hindi and English balanced because the target use case is Indian Hinglish rather than English-only speech.

7 Data engineering

The dataset is large, so I did not download everything into memory. Instead, I processed it piece by piece: pull a parquet shard, iterate a row group, select Hindi/English rows, decode the audio, resample to 16 kHz, create an 8-second window, save it as a WAV, and update a manifest. Once a shard was processed, I removed the local shard copy.

The manifest kept the audio path, language, endpoint label, filler labels, and dataset information for every sample. This made the data preparation resumable — if the notebook stopped halfway through, I could continue instead of starting again.

8 Audio preprocessing

All audio is converted to mono, 16 kHz, 8 seconds. If the audio is longer than 8 seconds, I keep the **last 8 seconds**; if it is shorter, I pad it at the beginning.

This is an important choice. Turn detection is mainly interested in *what happened just before the user stopped speaking* — so for a long utterance, the recent audio is more useful than an arbitrary early section. The official Smart Turn project also uses up to 8 seconds of audio and recommends retaining the most recent context when needed.

9 Why Whisper Tiny

I chose `openai/whisper-tiny` as the audio backbone. I did not need the Whisper decoder because I am not trying to transcribe the audio — I only need the encoder to answer "what is happening in this speech?" Whisper already provides a useful representation of speech, so instead of building a speech encoder from zero, I use the pretrained encoder and add a small task-specific classification head.

10 The model

I use three types of information.

A. Full audio context. The complete Whisper representation goes through a small CNN and attention pooling, giving the model the overall context of the utterance. “I want to book a flight to Delhi.” is different from “I want to book a flight to…” even if both have a pause at the end.

B. Recent audio. I separately process roughly the final 2 seconds, since the final part of the utterance is especially important for deciding whether someone has finished. The model gets both “what happened during the whole utterance” and “what happened immediately before the pause.”

C. Simple acoustic features. Five inexpensive audio features: onset strength, average energy, final energy relative to average energy, approximate pitch trend, and amount of active speech. These aren’t meant to replace Whisper — they give the model a few direct signals about things like energy dropping, speech activity, pitch movement, and changes near the end.

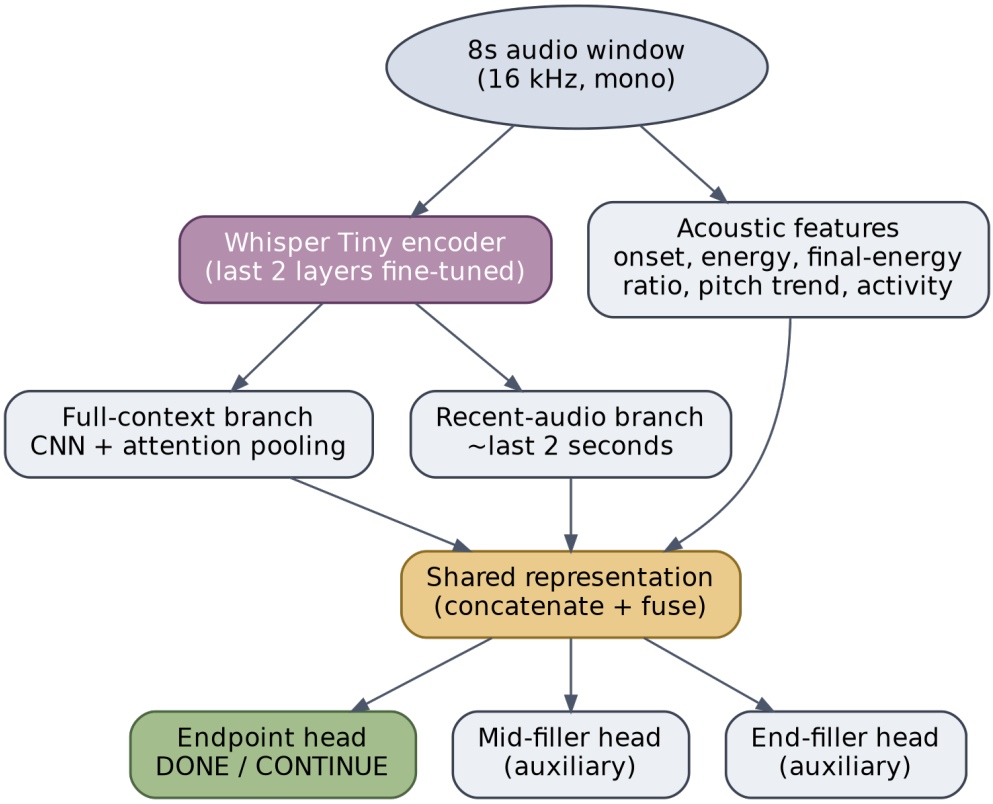


Figure 3: The turn detector combines full-context and recent-audio views from Whisper with a handful of hand-computed acoustic features, feeding a shared representation and three output heads.

11 Filler words

Filler words are particularly important for this problem. Examples: “umm…”, “uh…”, “matlab…”, “haan…”, “like…”. A filler does not necessarily mean the user has finished.

The dataset also contains labels related to fillers in the middle and at the end of an utterance, so I used these as **additional training signals**. The model has three heads sharing one representation — endpoint, mid-filler, and end-filler. The endpoint prediction remains the main task; the filler predictions are auxiliary tasks that help the shared representation learn more about conversational speech.

12 Training strategy

I did not fine-tune the entire Whisper model — most of Whisper is frozen. Only the final two encoder layers are allowed to learn, along with my classification layers.

This was a deliberate choice. I wanted lower training cost, fewer trainable parameters, less risk of destroying the useful pretrained speech representation, and a model that remains relatively small.

Setting	Value
Backbone	Whisper Tiny
Input	8 seconds / 16 kHz
Batch size	16
Gradient accumulation steps	2
Epochs	3
Learning rate	3e-5
Optimizer	AdamW
Mixed precision	FP16

13 Train / validation / test

The 40,000 samples are divided 80% training, 10% validation, 10% test. The split is stratified using **language + endpoint label**, which helps keep Hindi/English and complete/incomplete examples reasonably balanced across the splits.

The test set is not used to choose the model. The best checkpoint is selected using **validation F1**.

14 Why F1

Accuracy alone can be misleading — if a model predicts "continue speaking" most of the time, it may still get a reasonable accuracy depending on the dataset. For this problem, both mistakes matter:

- **False positive** — model says "user finished" but the user was still speaking. The assistant may interrupt them.
- **False negative** — model says "user is still speaking" but they had actually finished. The assistant waits unnecessarily.

So I track precision, recall, F1, accuracy, and the confusion matrix.

15 Results

Final results, reported on the held-out test set:

Metric	Value
Accuracy	0.9350
Precision	0.9239
Recall	0.9472
F1	0.9354

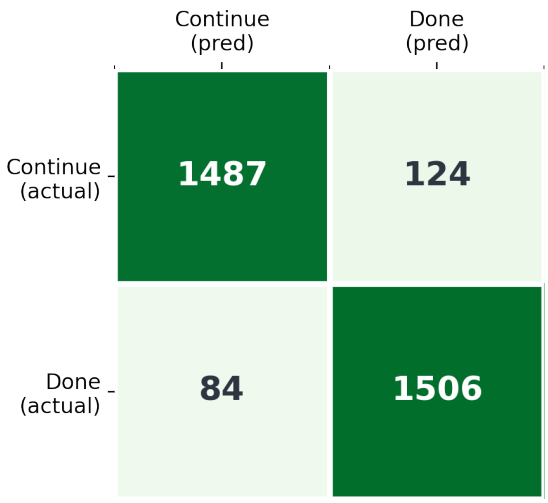


Figure 4: Confusion matrix on the held-out test set (counts).

Broken out by language:

Language	N	Accuracy	F1
Hindi	1,200	0.9450	0.9440
English	2,001	0.9290	0.9305

The model is slightly stronger on Hindi than English in this run, and recall is higher than precision overall — meaning false negatives (waiting a bit longer than needed) are somewhat more common than false positives (interrupting the user), which is the safer direction for a voice assistant to err toward. I'd keep this section factual and avoid claiming the model is production-ready based only on F1 — the next section covers the metrics that actually matter for a live product.

16 Demo

I built a simple Gradio demo. The user can speak into the microphone or upload an audio file. The model returns:

```
END OF TURN -- respond now
P(end of turn) = 0.87
```

or

```
STILL SPEAKING -- keep listening
P(end of turn) = 0.21
```

This gives a simple way to test the model without building the complete voice-agent stack.

17 A better product-level formulation

The current model uses Complete vs. Incomplete. But if I were taking this further for a real voice AI system, I would expand the problem to four conversational states:

State	Example	Action
Complete	“Book a flight to Delhi.”	Send to STT → LLM
Incomplete	“Book a flight to…”	Keep listening
Backchannel	“hmm”, “right”, “okay”	Usually don’t trigger full response
Wait	“wait a second”, “hold on”	Keep waiting

This is closer to what the user actually wants from a voice assistant. “Wait a minute, I need to check.” should not be treated exactly the same as “I need to check.” — the first explicitly asks the system to wait.

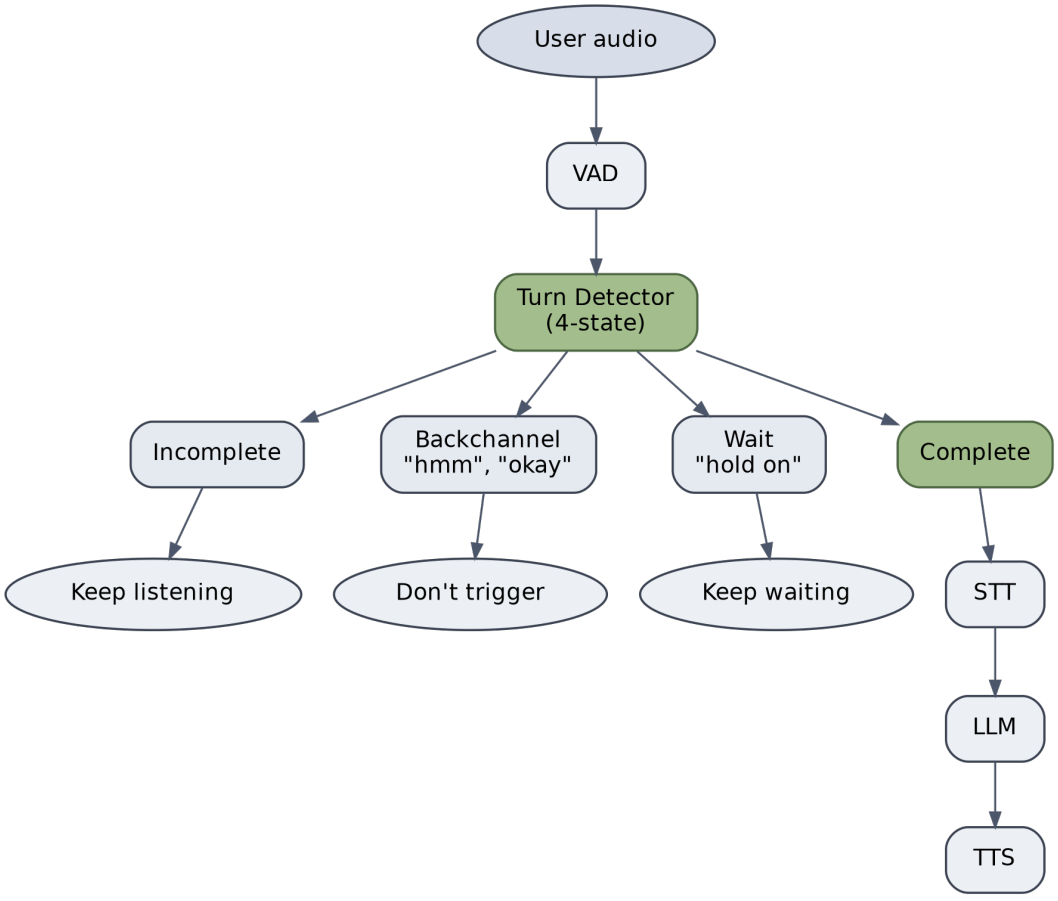


Figure 5: A four-state turn detector routes to different actions instead of a single binary decision — only “Complete” continues on to STT.

18 What I would build next

The current model is a good baseline, but there are several things I would test next.

1. Streaming inference. The current experiment uses an 8-second audio window. A real voice assistant would receive audio continuously — I would maintain a rolling audio buffer and periodically run the turn detector.

2. Optimize for latency. F1 is not enough for a voice product. I would measure how long after the user actually finishes the model decides — this is a much more useful product metric.

3. Measure interruptions. I would explicitly measure the premature response rate — how often the assistant responds while the user is still speaking. This may be more important than a small improvement in overall accuracy.

4. Threshold tuning. The current demo uses `P(end) >= 0.5`, but 0.5 is not necessarily the best production threshold. If interruptions are very expensive, we may want a higher threshold; if response latency is more important, a lower one.

5. Hard examples. I would create a hard-negative evaluation set with examples like "umm…", "yes but…", "wait…", "one second…", "actually…", "haan, but…" — these are much closer to the failures a real voice assistant will experience.

19 Final architecture

The system I would eventually aim for keeps each component doing one job:

```
VAD          -> Is there speech?
Turn model   -> Is the user done?
STT          -> What did they say?
LLM          -> What should we say?
TTS          -> How should we say it?
```

The turn detector is not trying to replace VAD or STT — it decides between four states, and only "Complete" turns continue on to the rest of the pipeline.

20 Conclusion

I approached this as a **voice-system problem rather than only a classification problem**. The first goal was to build a small and understandable baseline:

Whisper Tiny + recent audio context + simple acoustic features + filler information → turn completion prediction.

The model is intentionally small and only partially fine-tunes Whisper. The more important next step is not simply making the model bigger — it is measuring the things that actually affect a voice assistant:

- How quickly can we detect the end of a turn?
- How often do we interrupt users?
- How often do we wait unnecessarily?
- How well does it handle fillers and pauses?
- Does it behave consistently for Hindi, English, and Hinglish?

That is where I would take the project next.